

TD9

Protéger une application web en appliquant un codage sécurisé

Élève	Mohammed Fahad
Classe	BTS SIO SISR – 1ère Année
Établissement	Lycée Dominique Villars, Gap
Module	Cybersécurité – Codage sécurisé
Date	12/05/2026

Introduction

Ce compte rendu présente la réalisation du TD9 portant sur la protection d'une application web par l'application d'un codage sécurisé. Ce TP s'inscrit dans le cadre de la formation Cibeco sur la sécurisation des applications Web.

Durant ce TP, deux rôles ont été joués successivement :

- Le hacker : attaque d'un site Web interne (Mutillidae) pour afficher la liste des membres via une injection SQL.
- Le développeur : protection du site Web via des contre-mesures de codage sécurisé.

Objectifs du TP

- Comprendre et réaliser une attaque de type injection SQL.
- Analyser le code source vulnérable d'une application web.
- Appliquer des contre-mesures de codage sécurisé pour neutraliser l'attaque.
- Identifier les bonnes pratiques de développement sécurisé (OWASP).

Étape 1 – Préparation de l'environnement de travail

1.1 – Vérification et installation de VirtualBox

Avant de commencer, il convient de vérifier que la machine dispose d'au minimum 6 Go de mémoire vive, puis d'installer Oracle VirtualBox. Le fichier DELAGRAVE-LAB-THEME4.ova a été importé, créant les 4 machines virtuelles du labo :

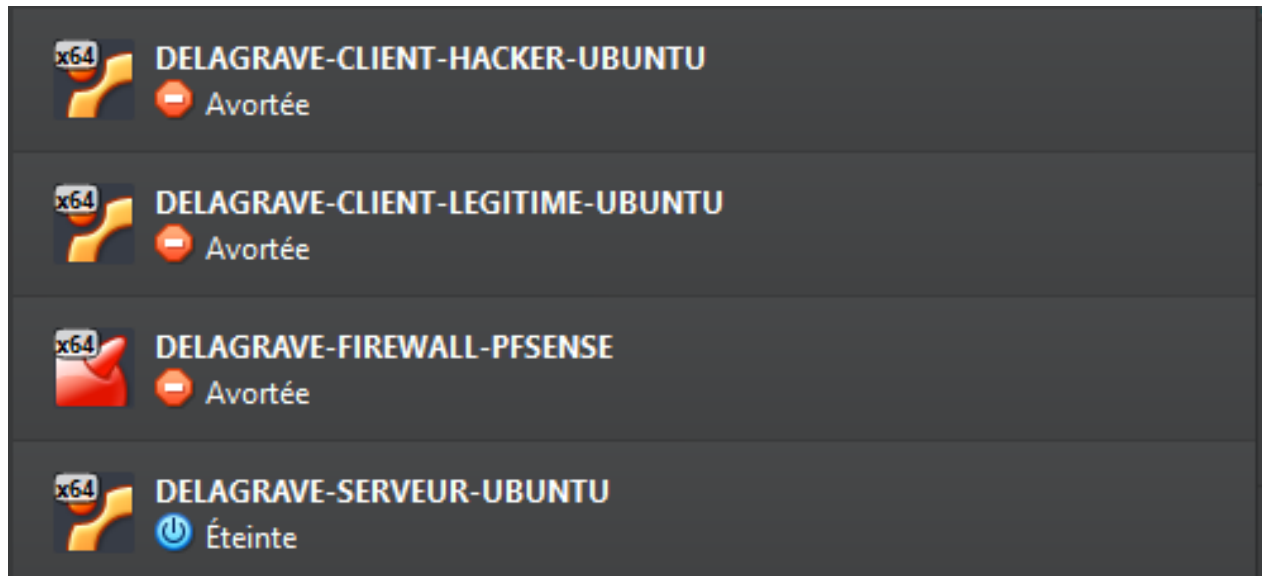


Figure 1 – Les 4 machines virtuelles importées dans VirtualBox

1.2 – Démarrage de la VM CLIENT-LEGITIME et configuration réseau

On démarre la VM DELAGRAVE-CLIENT-LEGITIME-UBUNTU. On suit ensuite les instructions de la vidéo LAB-THEME4-CONFIG.mkv présente sur le bureau pour configurer les cartes réseau des 4 machines.

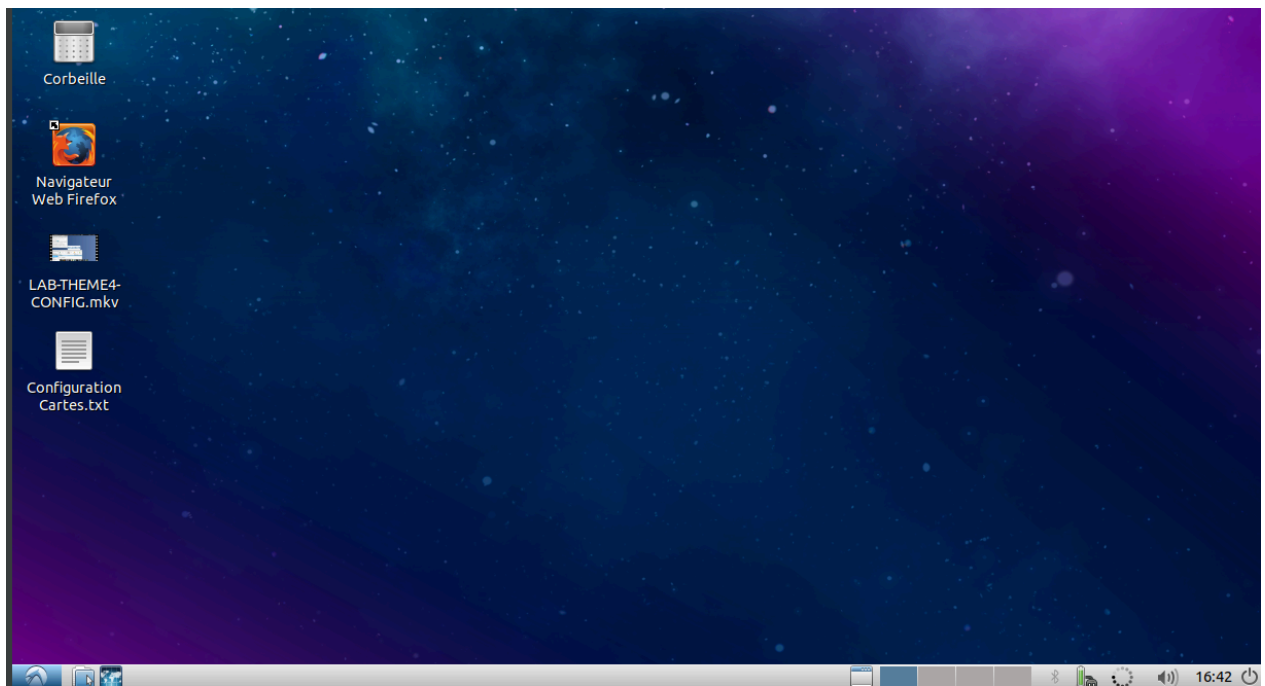


Figure 2 – Bureau de la VM DELAGRAVE-CLIENT-LEGITIME-UBUNTU

1.3 – Configuration manuelle de la carte réseau

La carte réseau est configurée en mode Manuel avec l'adresse IP 192.168.50.10, le masque /24 et la passerelle 192.168.50.254 (DNS : 8.8.8.8) :

Modification de Connexion filaire 4

Nom de la connexion :

Général | Ethernet | Sécurité 802.1X | DCB | Proxy | **Paramètres IPv4** | Paramètres IPv6

Méthode :

Adresses

Adresse	Masque de réseau	Passerelle
192.168.50.10	24	192.168.50.254

Serveurs DNS :

Domaines de recherche :

ID de client DHCP :

☐ Requiert un adressage IPv4 pour que cette connexion fonctionne

Figure 3 – Paramètres IPv4 de la connexion filaire (mode Manuel)

Après modification, on désactive puis réactive la carte réseau pour appliquer les paramètres :

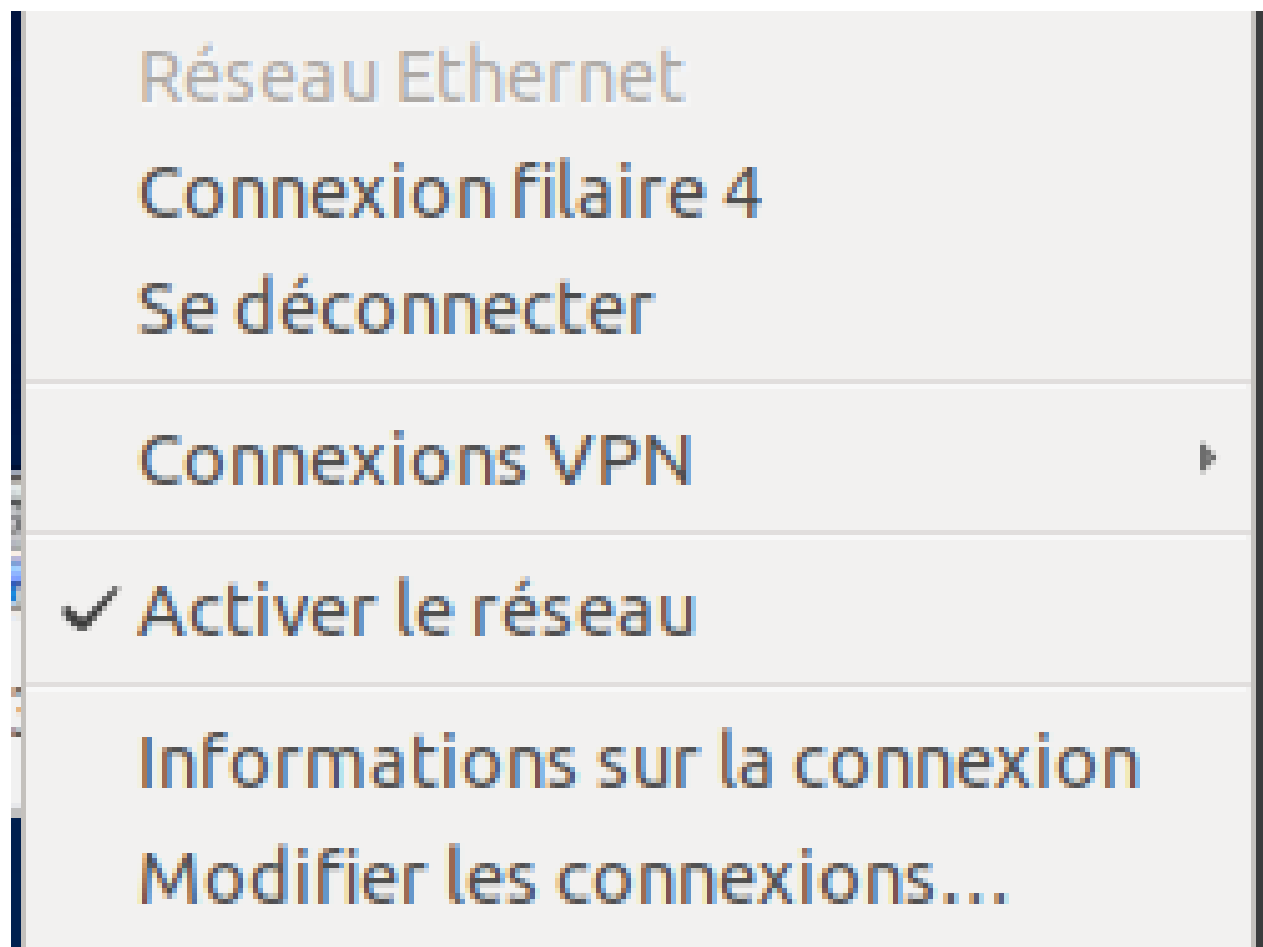


Figure 4 – Désactivation/réactivation de la carte réseau

La commande « ip a s » permet de vérifier que la configuration est bien prise en compte :

```
link/ether 08:00:27:d8:63:73 brd ff:ff:ff:ff:ff:ff
inet 192.168.50.10/24 brd 192.168.50.255 scope global noprefixroute enp0s3
    valid_lft forever preferred_lft forever
inet6 fe80::af3a:837c:9a40:62fa/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
```

Figure 5 – Vérification avec ip a s : adresse 192.168.50.10/24 confirmée

1.4 – Démarrage du serveur et du pare-feu

On démarre ensuite la VM DELAGRAVE-FIREWALL-PFSENSE (routeur du réseau) et la VM DELAGRAVE-SERVEUR-UBUNTU (qui héberge le site Mutillidae) :

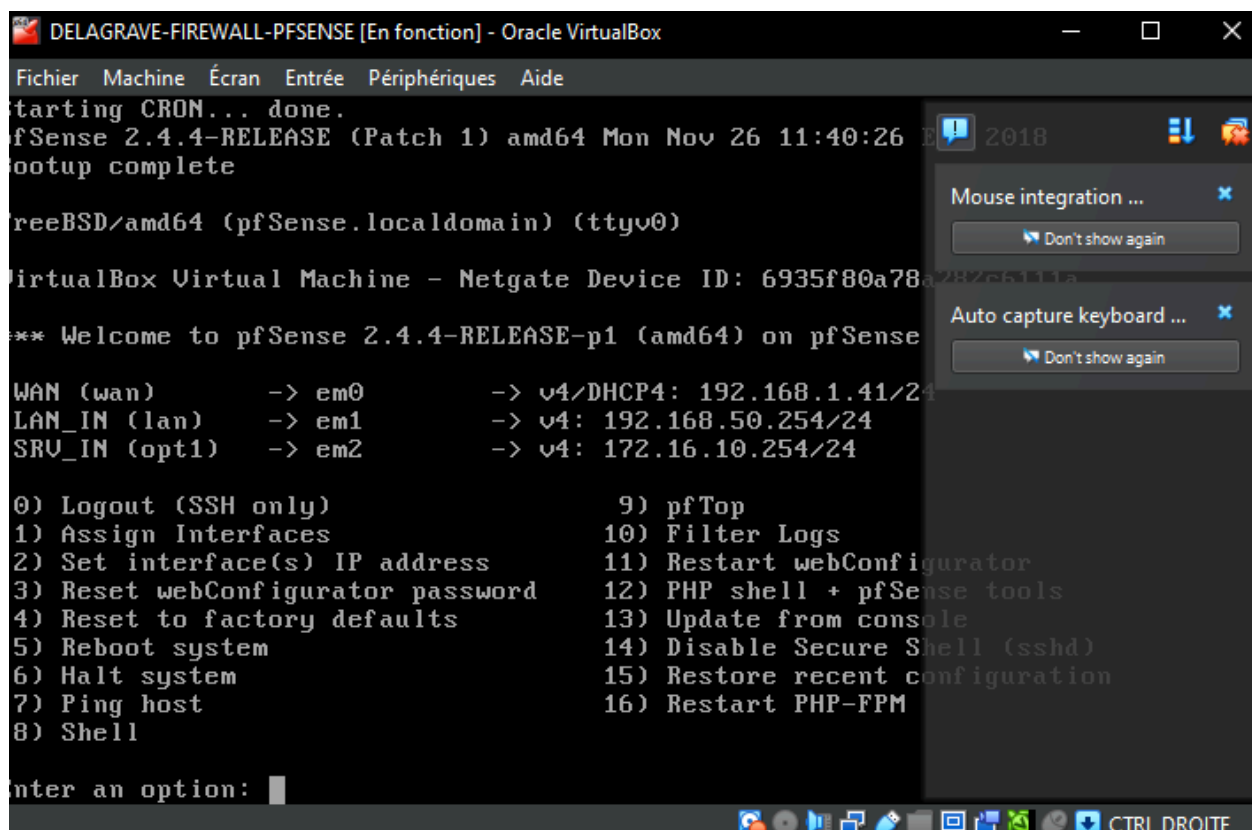


Figure 6 – VM DELAGRAVE-FIREWALL-PFSense démarrée (pfSense 2.4.4)

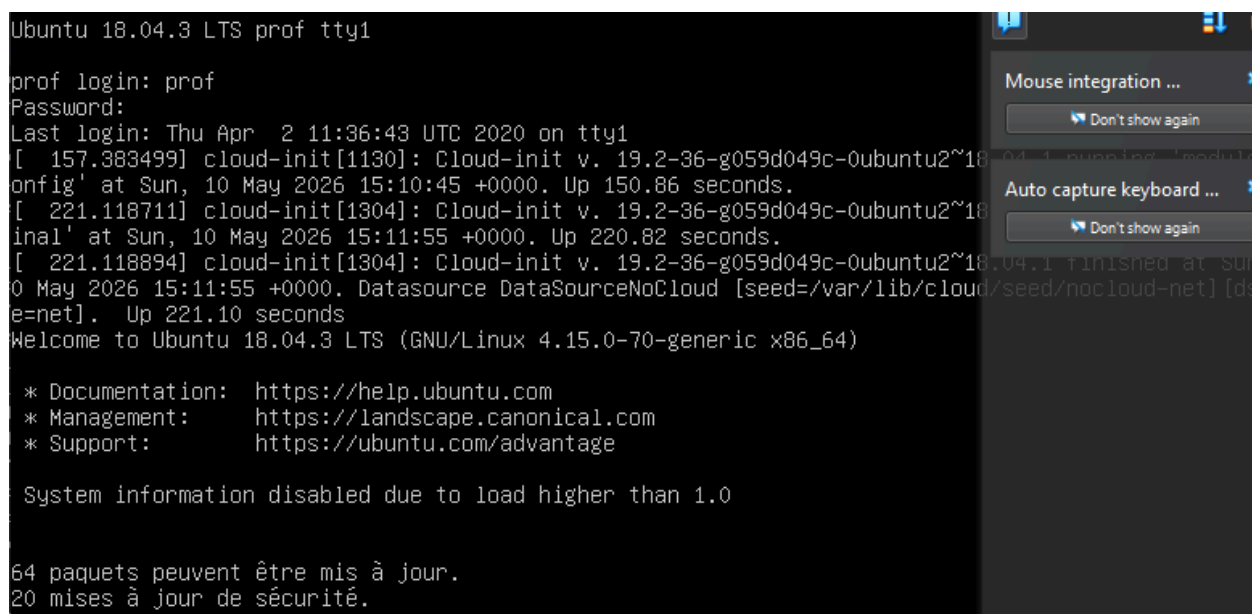


Figure 7 – VM DELAGRAVE-SERVEUR-UBUNTU démarrée (Ubuntu 18.04)

Machine	Rôle	Adresse IP
DELAGRAVE-SERVEUR-UBUNTU	Serveur Web – héberge Mutillidae	172.16.10.5

DELAGRAVE-CLIENT-LEG ITIME	Client légitime du réseau	192.168.50.10
DELAGRAVE-CLIENT-HA CKER	Machine de l'attaquant	192.168.50.20
DELAGRAVE-FIREWALL- PFSense	Routeur / Pare-feu pfSense	Passerelles

Étape 2 – Réalisation d'une attaque de type injection SQL

Question 5 – Injection SQL (Niveau de sécurité 0)

Rappel : Qu'est-ce qu'une injection SQL ?

Une injection SQL est une faille de sécurité qui permet d'injecter dans une requête SQL en cours d'exécution un morceau de code non prévu par le système. L'attaquant peut ainsi contourner l'authentification, exfiltrer des données ou modifier la base de données.

2.1 – Accès à Mutillidae depuis la VM Hacker

On démarre la VM DELAGRAVE-CLIENT-HACKER-UBUNTU et on accède au site Mutillidae à l'adresse 172.16.10.5/mutillidae/. Le niveau de sécurité est bien à 0 (Hosed) par défaut. On configure la carte réseau du hacker avec l'adresse 192.168.50.20/24 :

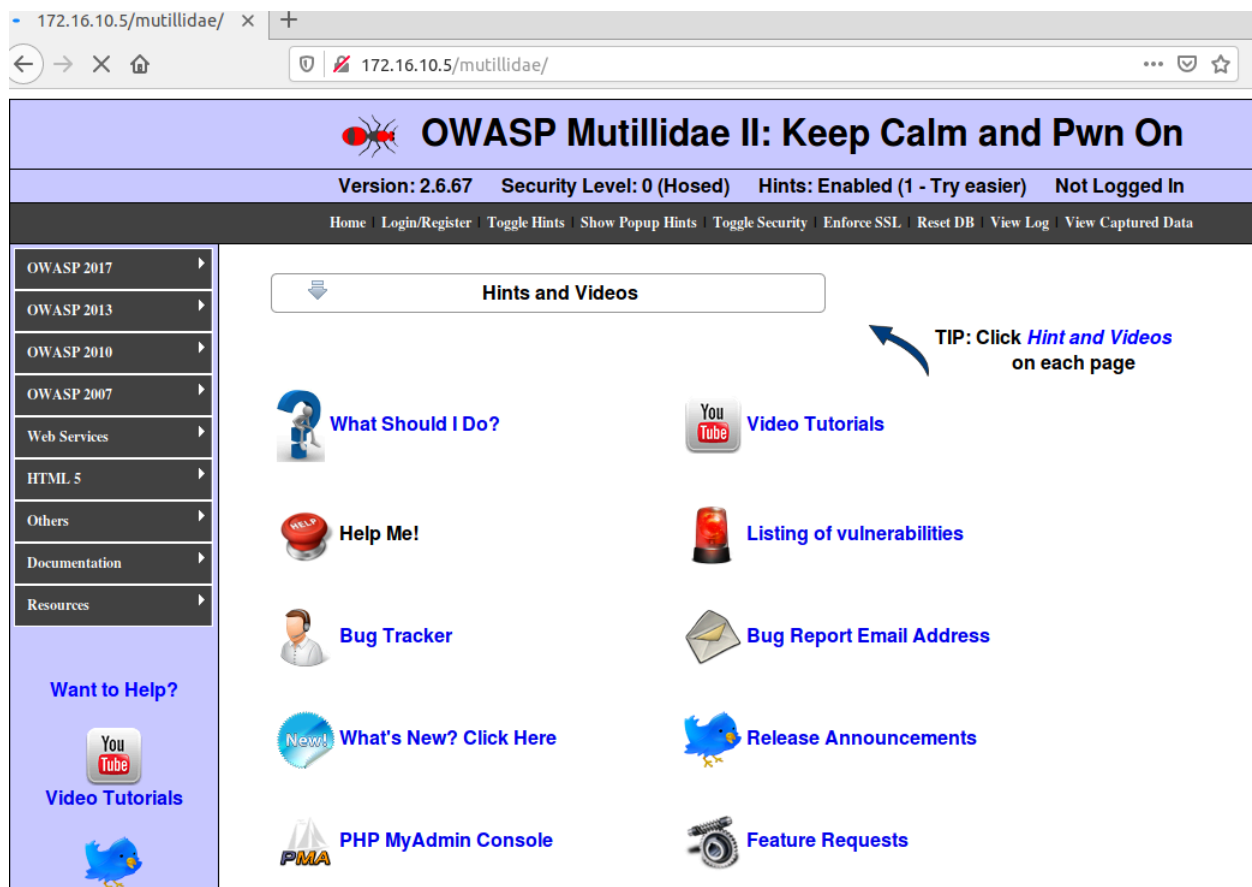


Figure 8 – Page d'accueil de Mutillidae (Security Level: 0 – Hosed)

Adresses		
Adresse	Masque de réseau	Passerelle
192.168.50.20	24	192.168.50.254

Figure 9 – Configuration IP de la VM Hacker : 192.168.50.20/24

2.2 – Navigation vers la page user-info.php et saisie du payload

Dans le menu de Mutillidae, on navigue vers : OWASP 2017 → Injection SQL → SQLi Extract Data → User Info (SQL).

On saisit la payload SQL suivante dans le champ Password :

```
' or 'a' = 'a

-- Cette condition est toujours vraie → tous les comptes sont retournés.
-- La requête backend devient : SELECT * FROM accounts WHERE
--   username='harry' AND password='' or 'a'='a'
```

Name

Password

View Account Details

Figure 10 – Formulaire avec le nom 'harry' et le payload SQL dans le champ Password

2.3 – Résultat de l'attaque

Results for "harry".23 records found.

Username=admin

Password=adminpass

Signature=g0t r00t?

Username=adrian

Password=somepassword

Signature=Zombie Films Rock!

Username=john

Password=monkey

Signature=I like the smell of confunk

Username=jeremy

Password=password

Signature=d1373 1337 speak

Username=bryce

Password=password

Figure 11 – Résultat : 23 comptes affichés (admin, adrian, john, jeremy, bryce...)

Constat – Injection SQL réussie (niveau 0)

L'attaque a fonctionné car le code PHP de user-info.php (niveau 0) n'effectue aucun contrôle sur les données saisies. La condition 'a'='a' étant toujours vraie, tous les enregistrements sont retournés (23 comptes avec usernames, mots de passe et signatures en clair). La vulnérabilité provient d'une absence totale de sanitisation des inputs, permettant au payload d'altérer la structure logique de la requête SQL backend.

Étape 3 – La contre-mesure de codage sécurisé

Question 6 – Passage au niveau de sécurité 5

On passe le niveau de sécurité à 5 (Server-side Security) via le bouton « Toggle Security » de Mutillidae, puis on rejoue la même injection SQL :

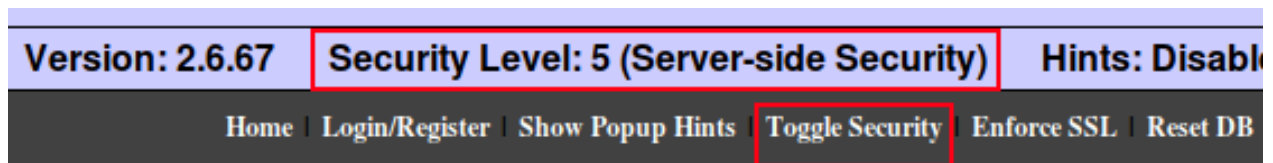


Figure 12 – Mutillidae : Security Level: 5 (Server-side Security)

Avec la même payload (' or 'a' = 'a), le résultat est maintenant :

Dangerous characters detected. We can't allow these. This all powerful blacklist will stop such attempts.

Much like padlocks, filtering cannot be defeated.

Blacklisting is l33t like l33tspeak.

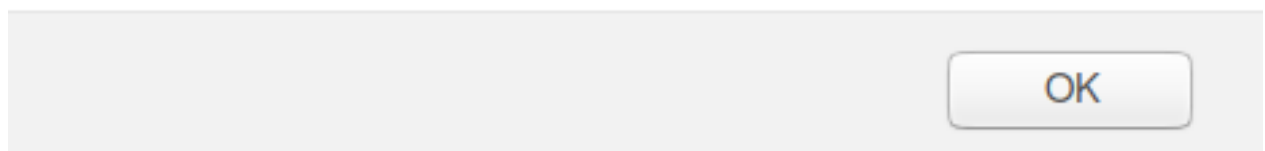


Figure 13 – Message : 'Dangerous characters detected. We can't allow these.'

Constat – Injection SQL bloquée (niveau 5)

L'injection SQL est complètement neutralisée. Le système détecte la présence de caractères dangereux (l'apostrophe) et affiche un message d'erreur générique sans révéler d'informations sensibles sur la base de données.

Question 7 – Analyse comparative du code source PHP

Sur la VM Serveur, on exécute « less /var/www/html/mutillidae/user-info.php » pour afficher le code source. On peut y comparer les deux versions :


```

<script type="text/javascript">
  <?php
    if($lEnableJavaScriptValidation){
      echo "var lValidateInput = \"TRUE\""; . PHP_EOL;
    }else{
      echo "var lValidateInput = \"FALSE\""; . PHP_EOL;
    }// end if
  ?>

  function onSubmitOfForm(/*HTMLFormElement*/ theForm){
    try{
      var lUnsafeCharacters = /[`~!@#%&*()-_+=\[\]{}\\|';",./<?]/;

      if(lValidateInput == "TRUE"){
        if (theForm.username.value.length > 15 ||
            theForm.password.value.length > 15){
          alert('Username too long. We dont want to allow too
many characters.\n\nSomeone might have enough room to enter a hack attempt. ');
          return false;
        }// end if

        if (theForm.username.value.search(lUnsafeCharacters) > -1 ||
            theForm.password.value.search(lUnsafeCharacters) > -1){
          alert('Dangerous characters detected. We can\'t allow
w these. This all powerful blacklist will stop such attempts.\n\nMuch like padlocks, filtering cannot
t be defeated.\n\nBlacklisting is 1337 like 1337speak. ');
          return false;
        }// end if
      }// end if(lValidateInput)

      return true;
    }catch(e){
      alert("Error: " + e.message);
    }// end catch
  }// end function onSubmitOfForm(/*HTMLFormElement*/ theForm)
:

```

Figure 16 – Code JavaScript : validation côté client, vérification longueur (15 car. max) et blacklist de caractères dangereux

Analyse : ce qui bloque l'attaque (niveau 5)

Deux mécanismes complémentaires sont activés au niveau 5 :

- Vérification de la longueur : le formulaire bloque toute saisie dépassant 15 caractères (username.length > 15 || password.length > 15 → return false).
- Liste noire (blacklist) : le code interdit l'utilisation de caractères spéciaux dangereux (apostrophes, guillemets, tirets...) via une regex appliquée à chaque champ avant soumission.

```

// Niveau 0 - NON SÉCURISÉ
$lEnableHTMLControls      = FALSE;
$lFormMethod              = 'GET';
$lEnableJavaScriptValidation = FALSE;
$lProtectAgainstMethodTampering = FALSE;
$lEncodeOutput            = FALSE;

// Niveau 5 - SÉCURISÉ
$lEnableHTMLControls      = TRUE;

```

```
$lFormMethod          = 'POST';    // plus difficile à manipuler
$lEnableJavaScriptValidation = TRUE;    // validation JS activée
$lProtectAgainstMethodTampering = TRUE; // protection anti-tampering
$lEncodeOutput         = TRUE;    // encodage des sorties
(anti-XSS)
```

Bonnes pratiques de codage sécurisé déduites

1. Ne jamais faire confiance aux données saisies par l'utilisateur.
2. Valider systématiquement la taille et le format de chaque champ de formulaire.
3. Filtrer les caractères dangereux via une blacklist (apostrophes, guillemets, tirets...).
4. Encoder les sorties (\$lEncodeOutput = TRUE) pour prévenir les attaques XSS.
5. Utiliser la méthode POST plutôt que GET pour les formulaires sensibles.
6. Utiliser des requêtes préparées (PDO/Prepared Statements) côté serveur.
7. Appliquer le principe du moindre privilège sur les comptes base de données.
8. S'appuyer sur les recommandations OWASP (Top 10 – A1: Injection).

Conclusion

Ce TD9 a permis d'appréhender concrètement la réalité des injections SQL, l'une des failles les plus répandues dans les applications web (classée A1 dans l'OWASP Top 10).

En jouant successivement le rôle du hacker puis du développeur, j'ai pu mesurer à quel point une application non sécurisée est vulnérable : en quelques secondes et avec une simple payload (' or 'a' = 'a'), il est possible d'extraire les 23 comptes de la base de données, avec usernames, mots de passe et signatures en clair.

La contre-mesure de codage sécurisé (niveau 5 de Mutillidae) démontre l'efficacité des bonnes pratiques : validation des entrées, blacklist de caractères interdits, encodage des sorties et méthode POST. Ces techniques, recommandées par l'OWASP, permettent de neutraliser complètement ce type d'attaque.

Bilan des compétences acquises

- Réalisation d'une attaque par injection SQL sur Mutillidae (OWASP).
- Lecture et comparaison de code PHP sécurisé vs non sécurisé.
- Identification des contre-mesures efficaces contre les injections SQL.
- Maîtrise des bonnes pratiques OWASP de codage sécurisé.